Python Beam Extraction and Monte Carlo Simulator (PY-BEMCS)

Bharat Singh Rawat

March 2026

Abstract

The primary inspiration behind PY-BEMCS originated during my doctoral research, where I worked extensively with ion sources and beam extraction systems. Throughout that time, I evaluated numerous simulation tools that utilized Finite Element Method (FEM) techniques to model beam extraction processes. However, the vast majority of these were proprietary commercial codes locked behind prohibitive license fees and recurring service charges.

My vision for PY-BEMCS is to break down these barriers by providing a powerful, fully open-source beam extraction simulator dedicated to non-commercial research by academics and students. A core design philosophy of this project is accessibility: it is engineered to be as user-friendly as possible, removing the steep learning curve of complex programming so that researchers can focus directly on the physics. As detailed in this document, the resulting engine utilizes a Hybrid Particle-in-Cell (PIC) / fluid approach to model plasma dynamics, coupled with Monte Carlo Collisions (MCC) for neutral interactions, and a finite-difference thermal model for structural degradation.

I officially initiated this project today (26/03/2026), on my 34th birthday, as a personal commitment to fostering accessible, open science for the next generation of researchers. This document details the physical models, numerical methods, and multiphysics couplings implemented in PY-BEMCS. The engine utilizes a Hybrid Particle-in-Cell (PIC) / fluid approach to model plasma dynamics, coupled with Monte Carlo Collisions (MCC) for neutral interactions, and a finite-difference thermal model for structural degradation.

Contents

1 Fundamental Domain & Scaling **2**

2 Electrostatics & Space Charge (Hybrid PIC) **2**

2.1 The Non-Linear Poisson Equation 2

2.2 Numerical Solver 3

3 Ion Kinematics & Injection **3**
3.1 Bohm Criterion Injection 3
3.2 Particle Push 3

4 Charge-Exchange (CEX) Collisions **4**

5 Grid Erosion & Structural Failure **4**
5.1 Sputtering Yield 4
5.2 In-Situ Remeshing 4

6 Secondary Electron Emission (SEE) **4**

7 Thermal Multiphysics **5**
7.1 Kinetic Heating 5
7.2 Radiative Cooling 5
7.3 Thermal Expansion 5

8 How to Cite This Model **5**

---

1 Fundamental Domain & Scaling

To simulate nanosecond-scale plasma physics alongside minute-scale thermal and erosion dynamics, the engine employs several numerical scaling techniques:

- **Macro-Particles:** Instead of tracking billions of individual ions, the engine tracks "macro-particles." Each macro-particle represents a specific number of real ions, defined by the weight W. This weight is dynamically auto-scaled based on the plasma density (n_0) to maintain an optimal particles-per-cell ratio.
- **Artificial Electron Mass:** Tracking true electrons requires femtosecond timesteps due to their high velocity. To allow for a larger standard timestep ($\Delta t = 10^{-9}$ s) while still resolving electron backstreaming trajectories, the explicitly tracked secondary electrons are assigned an artificial mass ($m_e = m_{Xe}/100$).
- **Thermal Acceleration Factor:** Thermal dynamics happen millions of times slower than ion transits. A multiplier (a_{th}) scales the specific heat and radiative cooling equations so that grid temperatures and deformations can be observed in a reasonable number of simulation iterations.

2 Electrostatics & Space Charge (Hybrid PIC)

2.1 The Non-Linear Poisson Equation

The electric potential $V(x, y)$ is solved on a 2D Cartesian grid. Instead of tracking bulk plasma electrons as particles, the engine treats them as a massless

fluid following the Boltzmann relation. The governing Poisson equation is:

$$\nabla^2 V = -\frac{\rho_{total}}{\epsilon_0} = -\frac{\rho_{ion} + \rho_e}{\epsilon_0} \tag{1}$$

Where ρ_{ion} is gathered directly from the positions of the explicitly simulated macro-ions using nearest-grid-point weighting. The electron charge density ρ_e assumes a Maxwell-Boltzmann distribution referencing the upstream plasma potential (V_{plasma}) and electron temperature ($T_{e,up}$):

$$\rho_e(V) = -qn_0 \exp\left(\frac{\min(V, V_{plasma}) - V_{plasma}}{T_{e,up}}\right) \tag{2}$$

2.2 Numerical Solver

Because the boundaries change dynamically during erosion but remain static for hundreds of steps at a time, the engine pre-computes the LU Factorization of the Laplacian sparse matrix. To handle the non-linearity of the Boltzmann electrons, the solver uses Picard Iterations with successive under-relaxation ($\omega = 0.2$) to ensure the potential field converges smoothly without numerical oscillation.

3 Ion Kinematics & Injection

3.1 Bohm Criterion Injection

Xenon ions (Xe^+) are injected at the upstream boundary. To satisfy the physical requirements of sheath formation, ions must enter the simulation domain at or above the Bohm velocity:

$$v_{Bohm} = \sqrt{\frac{qT_{e,up}}{m_{Xe}}} \tag{3}$$

The total ion current I_{ion} entering the domain is scaled by the effective injection area and the pre-sheath density drop coefficient (0.61):

$$I_{ion} = 0.61 \cdot q \cdot n_0 \cdot v_{Bohm} \cdot A_{inject} \tag{4}$$

3.2 Particle Push

Particles are accelerated by the local electric field $\vec{E} = -\nabla V$. The field is interpolated continuously from the discrete grid to the exact particle coordinates. The kinematic update uses an explicit integration scheme:

$$\vec{v}(t + \Delta t) = \vec{v}(t) + \frac{q}{m}\vec{E}\Delta t \tag{5}$$

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t + \Delta t)\Delta t \tag{6}$$

4 Charge-Exchange (CEX) Collisions

Fast beam ions can interact with slow background neutral Xenon gas, exchanging an electron. The relative velocity g between the ion and neutral is approximated as the ion's magnitude of velocity. The empirical cross-section σ_{CEX} for Xe^+ on Xe is calculated as:

$$\sigma_{CEX}(g) = \left(-0.8821 \ln(g) + 15.1262\right)^2 \times 10^{-20} \ [\mathrm{m}^2] \tag{7}$$

The probability P_{CEX} of a collision occurring for any given macro-ion in a single timestep Δt is evaluated using the Monte Carlo method:

$$P_{CEX} = 1 - \exp\left(-n_n \sigma_{CEX}(g) g \Delta t\right) \tag{8}$$

If a collision occurs probabilistically, the ion's velocity is instantly reassigned to match the thermal velocity of the background neutral gas.

5 Grid Erosion & Structural Failure

5.1 Sputtering Yield

Physical damage is quantified by an empirical sputter yield $Y(E)$, representing Molybdenum atoms removed per incident Xenon ion. The threshold energy for sputtering is roughly 30 eV. The yield curve is modeled as:

$$Y(E) = \begin{cases} 1.05 \times 10^{-4} \left(E - 30\right)^{1.5} & E > 30 \text{ eV} \\ 0 & E \leq 30 \text{ eV} \end{cases} \tag{9}$$

5.2 In-Situ Remeshing

Damage is accumulated in a 2D scalar field. When a grid cell's accumulated damage exceeds the defined structural threshold, the cell is flagged as broken. The engine immediately removes this cell from the solid boundary mask, rebuilding and re-factorizing the Sparse Laplacian matrix to allow plasma to flow through the newly eroded hole.

6 Secondary Electron Emission (SEE)

High-energy ion impacts on the grids can eject electrons from the metal lattice. The empirical yield γ_{SEE} (electrons emitted per incident ion) is modeled linearly above a baseline:

$$\gamma_{SEE}(E) = \max\left(0, \min\left(1, 0.05 + 10^{-4} E\right)\right) \tag{10}$$

If an emission event occurs, macro-electrons are spawned slightly offset from the grid wall with a random thermal energy of roughly 2.0 eV. They are then explicitly tracked by the PIC engine.

7 Thermal Multiphysics

7.1 Kinetic Heating

Kinetic energy from impacting macro-particles is deposited into the grid geometry as heat. The thermal energy ΔQ deposited per timestep is calculated from the Xenon mass and the macro-weight W:

$$\Delta Q = \frac{1}{2} m_{Xe} |\vec{v}|^2 W \tag{11}$$

This energy increases the local cell temperature based on the volumetric heat capacity C_{cell}:

$$\Delta T_{heat} = \left(\frac{\Delta Q}{C_{cell}} \right) a_{th} \tag{12}$$

7.2 Radiative Cooling

Grids dissipate heat into the vacuum via Stefan-Boltzmann radiation. The temperature reduction per timestep for boundary cells is:

$$\Delta T_{cool} = \frac{\epsilon \sigma_{SB} A_{cell}}{C_{cell}} \left(T^4 - 300^4 \right) \Delta t \cdot a_{th} \tag{13}$$

7.3 Thermal Expansion

As the accelerator grid temperature (T_a) rises above the 300K baseline, linear thermal expansion alters the physical gap distance d_{gap} between the screen and accelerator grids.

$$\Delta d_{gap} = \alpha_{moly} \cdot d_{gap} \cdot (T_a - 300) \tag{14}$$

If the gap distance changes by more than 0.05 mm, the engine recalculates the mesh geometry to capture the effects of thermal grid warping.

---

8 How to Cite This Model

If you use this PY-BEMCS or its underlying physics engine in your research, please cite the repository as follows:

Plain Text:
Bharat Singh Rawat. (2026). *Python Beam Extraction and Monte Carlo Simulator (PY-BEMCS)* [Source code]. GitHub. https://github.com/Bharat26031992/PY-BEMCS

BibTeX:

```
@software{PY-BEMCS,
  author       = {Bharat Singh Rawat},
  title        = {Python Beam Extraction and Monte Carlo Simulator (PY-BEMCS)},
  year         = {2026},
  publisher    = {GitHub},
  journal      = {GitHub repository},
  howpublished = {\url{https://github.com/Bharat26031992/PY-BEMCS}},
}
```